

CREANDO UN PROYECTO

GUÍA WINDOWS

DESARROLLO WEB EN ENTORNO SERVIDOR

1. REQUISITOS PREVIOS

Nota para Windows: Los comandos de esta guía están adaptados para **PowerShell / CMD** en Windows. Lee atentamente las notas y avisos para evitar errores comunes de variables de entorno y permisos.

1. Antes de empezar, vamos a asegurarnos de tener instaladas las siguientes aplicaciones:

1. **VSCode**

2. **Python**

⚠ Al instalar Python en Windows es **obligatorio** marcar la casilla "**Add python.exe to PATH**". Si ya lo instalaste sin marcarla o la terminal no reconoce **python**, vuelve a ejecutar el instalador y selecciona **Modify** para activarla.

2. Vamos a crear una carpeta llamada **Backend** para alojar el código de nuestro proyecto. Abre la carpeta con VSCode (o en la terminal: `mkdir Backend; cd Backend`).

3. Dentro de VSCode instalamos los siguientes plugins:

1. Python Environments
2. Pylance
3. Python
4. Python Debugger
5. HTML CSS Support
6. Django (Baptiste Darthenay)

4. Dentro de VSCode y con la carpeta Backend abierta, abrimos la terminal e instalamos el paquete **Pipenv**:

```
$ pip install pipenv
```

💡 Si al ejecutar scripts en PowerShell recibes un error de permisos o directiva de ejecución, ejecuta:
`Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

5. Pipenv es un entorno virtual para desarrollo de aplicaciones de Python. Creamos el entorno virtual e instalamos librerías:

```
$ python -m pipenv install requests
```

💡 Usar **python -m pipenv** asegura que Windows encuentre Pipenv directamente, evitando el error de comando no reconocido si la carpeta Scripts aún no se ha indexado en el PATH.

6. Si lo hemos hecho correctamente deberíamos tener dos archivos (Pipfile y Pipfile.lock) dentro de nuestra carpeta de proyecto. Ahora lanzamos el entorno virtual:

```
$ python -m pipenv shell
```

2. DJANGO

Nota: Vamos a proceder a crear un proyecto con Django. Intenta tener una idea clara de qué proyecto vas a intentar desarrollar. Recuerda que el nombre del proyecto, la documentación del código y el contenido de la web deben estar en inglés.

7. El prompt de la terminal habrá cambiado ligeramente (mostrando el entorno virtual activo). Instalamos Django:

```
$ pip install django
```

8. Crea un archivo `__test__.py` desde el editor de VSCode para hacer pruebas de código. Vamos a verificar la instalación de Django y que estamos usando el intérprete correcto. (Para cambiar de intérprete tenemos el atajo **CTRL+SHIFT+P** y luego buscamos *Python: Select Interpreter* seleccionando el entorno Pipenv).

```
__test__.py
import django
print(django.get_version())
```

9. Ejecuta el archivo `__test__.py` y asegúrate de que todo funciona correctamente:

```
$ python __test__.py
```

10. Ahora vamos a arrancar con lo interesante. Vamos a crear el proyecto de Django con el siguiente comando:

```
$ django-admin startproject NOMBRE_DEL_PROYECTO
```

11. Se habrán creado los archivos y la estructura del proyecto. Accedemos a la carpeta del proyecto y ejecutamos `manage.py` para lanzar el servidor:

```
$ cd NOMBRE_DEL_PROYECTO
$ python manage.py runserver
```

12. Intenta acceder a tu web desde el navegador en: **`http://localhost:8000`** o **`http://127.0.0.1:8000`**

13. Si la web funciona ya puedes subir este repositorio a GitHub y hacer un merge hacia Staging desde Development:

```
$ git init
$ git add .
$ git commit -m "Initial Django project setup"
```